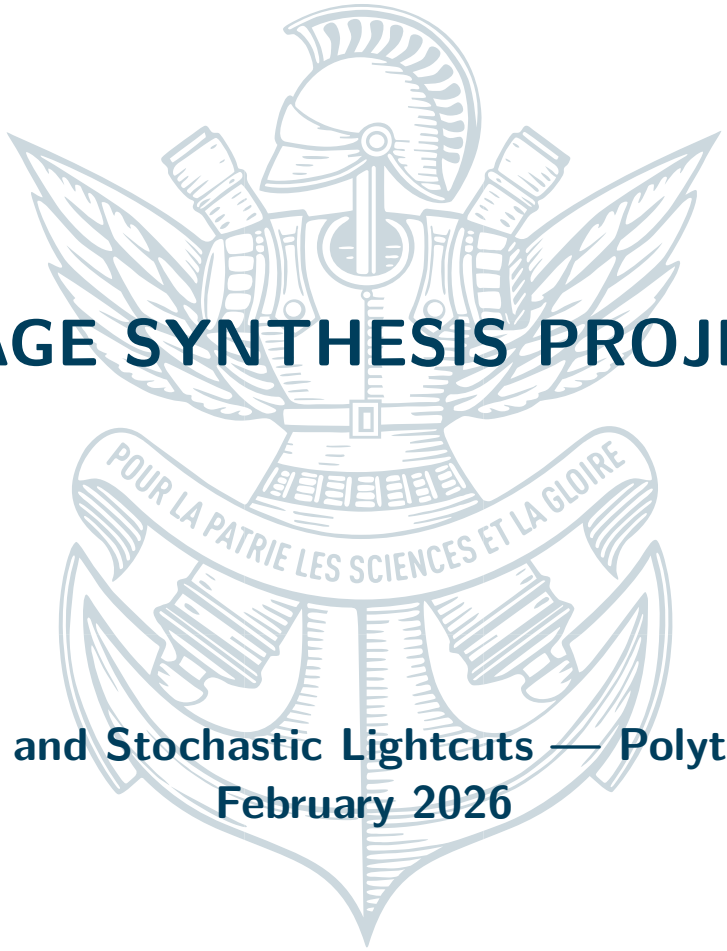


IMAGE SYNTHESIS PROJECT

Lightcuts and Stochastic Lightcuts — Polytechnique, February 2026

February 24, 2026

Nathan Duboisset

1 INTRODUCTION

This project implements three many-light rendering algorithms on top of a WebGPU ray-tracer built during the course assignments: **Lightcuts** [1], **Stochastic Lightcuts** [2, 3], and a **tile-precomputed variant** inspired by real-time stochastic lightcuts [4].

The test scene is a planar floor with a ram mesh, shaded with a Cook-Torrance microfacet BRDF. To study the algorithms in isolation, the scene is lit by **1600 point lights arranged in a panel above the objects** rather than full VPLs — this keeps the light count fixed and makes timing differences and visual artefacts clearly observable. All algorithms run in WGSL compute shaders; the tree data structures are built on the CPU and uploaded as GPU storage buffers.

2 LIGHTCUTS

2.1 Implementation

Lightcuts [1] clusters lights into a binary tree and greedily selects a per-pixel *cut* of K nodes, evaluating only one representative per node instead of all n lights.

Tree construction (`src/lightcutTree.ts`). Nodes are merged bottom-up with a priority queue (`js-priority-queue`) using:

$$c(A, B) = \|\mathbf{p}_A - \mathbf{p}_B\|^2 + \mathrm{vol}(\mathrm{AABB}(A \cup B))$$

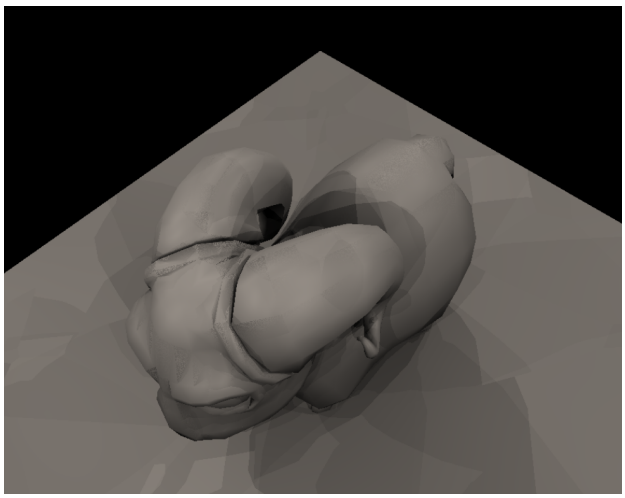
Each node stores an intensity-weighted representative and union AABB, serialised into 16-float structs uploaded to binding 8.

GPU cut construction (`computeRadianceLightcuts` in `shaders.wgsl`). Each shader invocation iteratively splits the highest-error node ($K_{\max} = 32$) using:

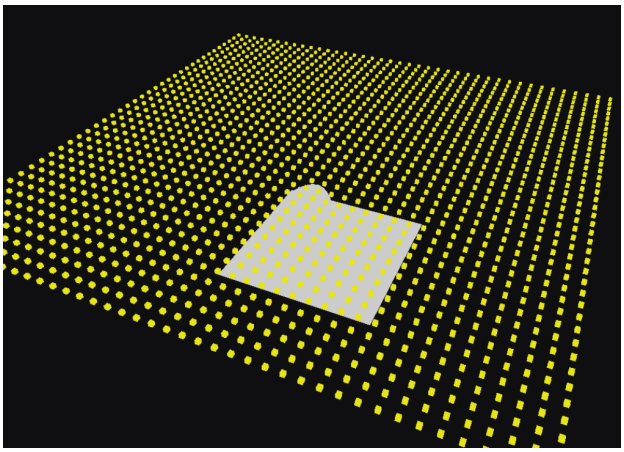
$$\varepsilon(v, \mathbf{x}) = \frac{I_v \cdot \mathrm{diag}(v)}{\|\mathbf{x} - \mathbf{p}_v\|^3}$$

Each final cut node contributes one BRDF evaluation and one shadow ray.

2.2 Results

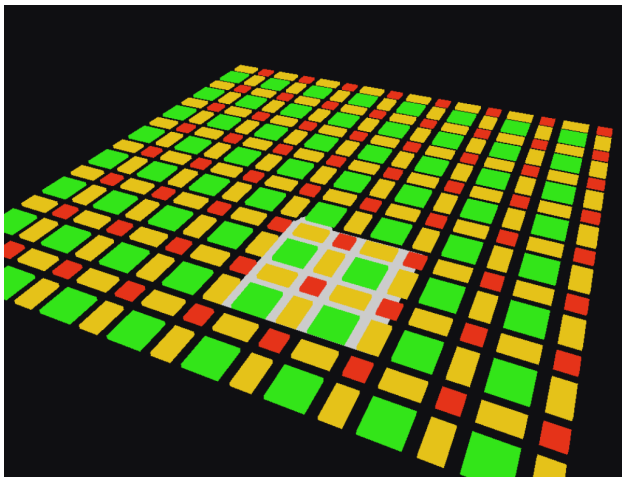


(a) Lightcuts render — 1600 lights, $K = 32$. We can see visual artefacts

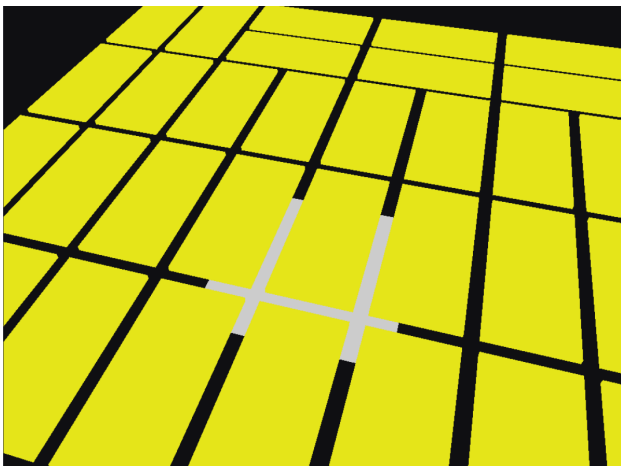


(b) Lightcut tree — deepest level bounding boxes.

Figure 1: Lightcuts output and tree visualisation.



(a) Near bottom of the tree cuts



(b) Near top of the tree cuts

Figure 2: Tree visualisation at two depths — AABB bounding boxes rendered in a separate canvas.

3
STOCHASTIC LIGHTCUTS

3.1 Implementation

Stochastic Lightcuts [2] keeps the same cut but replaces the deterministic sum with a Monte Carlo estimator: for each cut node, one leaf is importance-sampled and its contribution rescaled by the inverse probability.

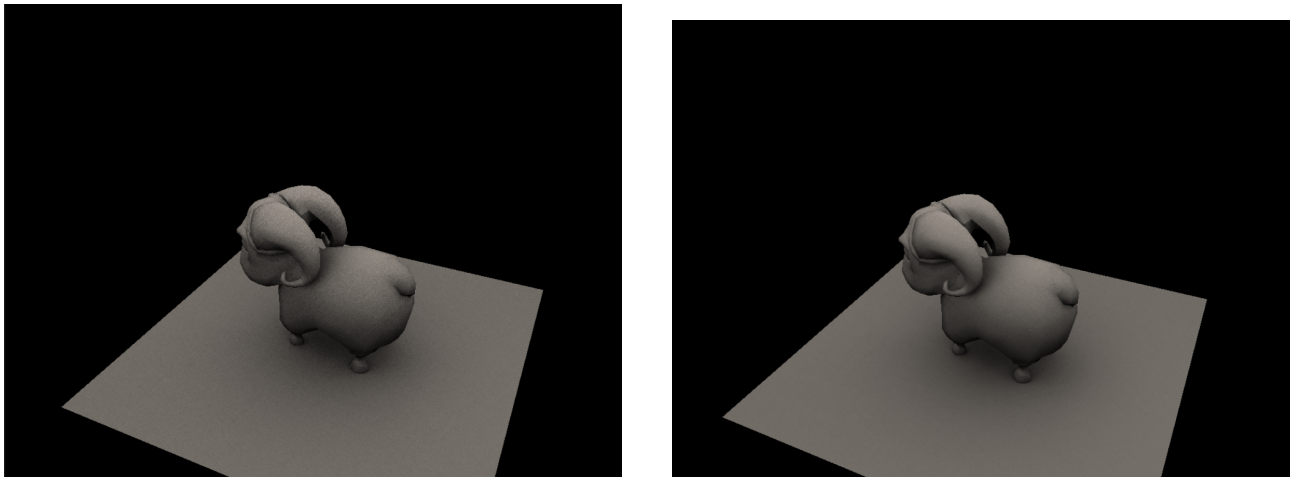
HIS — `selectLight()` in `shaders.wgsl`. The traversal weight at each node is:

$$w(v) = \begin{cases} I_v / d_{\min}^2 & d_{\min} > \mathrm{diag}(v) \\ I_v & \text{otherwise} \end{cases}$$

biasing toward nearby bright clusters; the sampled leaf contributes $f(\ell) \cdot V(\ell)/p$.

Temporal accumulation. The frame counter seeds a per-pixel PCG hash for independent samples each frame. Frames are accumulated additively and divided by the pass count. Figures 3a and 3b compare 16 and 64 passes.

3.2 Results



(a) 16 accumulation passes.

(b) 64 accumulation passes.

Figure 3: Stochastic Lightcuts — 1600 lights, $K = 32$, increasing accumulation.It smoothens a bit more, approaching the full ray-traced viz

Note that at that rate(64 accumulation), the time necessary to compute is quite large and close to the time needed to a full raytrace render with all the lights. This is not surprising because the number of lights here is quite small (big enough to observe results, small enough to have reasonable computation times without the webGPU throwing errors). But the complexity of the stochastic lightcuts approach scales with the logarithm of the number of light, thus it will be far more efficient with more lights, with VPL for example.

4
TILE-PRECOMPUTED STOCHASTIC LIGHTCUTS

4.1 Implementation

Per-pixel cut construction is the dominant cost of Stochastic Lightcuts. Following Lin & Yuksel [4], one cut is precomputed per screen tile on the CPU (`buildCutsForTiles` in `src/lightcutTree.ts`): a ray through the tile centre gives a representative world position; a max-heap builds a cut of up to K nodes in $O(K \log n)$ and packs it into a flat `u32` buffer (stride $K + 1$, binding 9) uploaded before each frame.

On the GPU (`computeRadianceRealtimeStochasticLightcuts`), each pixel looks up its tile cut with a single integer division and runs HIS only. `selectLightRT()` uses the average of $p_{\min}/p_{\max}$ bounds so one cut serves every pixel in the tile.

The method is not truly real-time in a browser (CPU build + JS upload add latency), but the in-shader cost is noticeably lower than per-pixel cut construction (Table 1).

5
RESULTS AND PERFORMANCE

5.1 Performance

All timings are wall-clock averages on a Nvidia RTX 4060, 1600 lights, $K = 32$. Are noted the number of passes to smoothen the stochastic part.

Table 1: Average render time per frame — 1600 point lights, ram scene.

Method	**ms / frame**	**Notes**
Brute-force RT	24000	reference
Lightcuts ($K = 32$)	400	with artifacts
With 4 passes		
Stochastic LC ($K = 32$)	1500	nearly no aliasing
Tile-precomputed SLC(compute tiles of 3x3)	1300	CPU upload incl.

6 CONCLUSION

Three many-light algorithms were implemented in WebGPU/WGSL on top of the course ray-tracer. Lightcuts delivers an image in a single pass at a fraction of the brute-force cost, but includes visual artifact that are really important. Stochastic Lightcuts is faster per frame and converges through temporal accumulation really quickly. The tile-precomputed variant further reduces in-shader cost, validating the Lin & Yuksel 2020 approach even within the constraints of a browser runtime.

REFERENCES

[1] B. Walter, S. Fernandez, A. Arbree, K. Bala, M. Donikian, D. P. Greenberg. Lightcuts: A scalable approach to illumination. *ACM SIGGRAPH 2005 Papers*, pp. 1098–1107, 2005.

[2] C. Yuksel. Stochastic Lightcuts. *High-Performance Graphics (HPG)*, 2019. Wolfgang Straßer Best Paper Award.

[3] C. Yuksel. Stochastic Lightcuts for Sampling Many Lights. *IEEE Transactions on Visualization and Computer Graphics*, 2020.

[4] D. Lin, C. Yuksel. Real-Time Stochastic Lightcuts. *Proc. ACM Comput. Graph. Interact. Tech. (I3D 2020)*, 3(1), 2020. Best Paper Award.